Machine-Checked Complexity of Decision-Table Completeness in Lean

Carlos A. Russell[1*]

[1*]Independent Researcher, Buenos Aires, Argentina.

Corresponding author(s). E-mail(s): carlos.a.russell@gmail.com;

Abstract

A decision table is complete when every combination of its declared condition values that the domain constraints allow is covered by a row. We prove in Lean 4, over multi-tape Turing machines and with no unproved assumptions beyond the definitions themselves, that for finite-domain decision tables with clause constraints and rows that are complete condition tuples, deciding whether some combination is missing is NP-complete, deciding completeness is coNP-complete, and counting the missing combinations is #P-complete, all already for Boolean domains and empty tables. An adequacy theorem ties these word problems to the set-based definitions of an existing formalisation of decision-table completion, so the results speak about the original objects. Complementary polynomial-time results locate the hardness: all three problems are polynomial without constraints, and with a fixed number of attributes, where one machine runs within $c\,(|x|+1)^{en+e}$ on instances with n attributes, for some constants c and e. For monotone two-literal constraints, counting the missing combinations remains #P-complete under reductions with a single oracle call; for this we give a new formal proof, in the same Turing-machine model, of a one-call version of Valiant's theorem on #Monotone-2SAT. We also report how the development was engineered and how an AI system was used to write it. The classification was first argued on paper in the author's earlier work, which is under review; every proof here is self-contained.

Keywords: decision tables, completeness, coNP-completeness, #P-completeness, Lean 4, Cook–Levin theorem

1 Introduction

Decision tables state business rules, regulations and clinical guidelines as rows of condition values [1]. A table is *complete* when every combination of declared condition values that the domain constraints allow is covered by some row; the uncovered combinations form its *missing set* $M(T_0)$. Completeness checking is a staple of decision-table practice, from early expert-system checkers [2] to the analysers of the DMN standard [3], which report missing rules. Counting them matters too: one missing edge case and a table that misses half of its valid inputs are both "incomplete", and $|M(T_0)|$ is what tells them apart.

Russell [4], in a paper under review, re-examined a decision-table completion method over declared finite domains and argued, on paper, that completeness under clause constraints is coNP-complete and that counting $|M(T_0)|$ is #P-complete (Proposition 5 there). The present paper does not depend on that argument: every proof it gives is its own. The accompanying Lean development, latc-check [5], formalised the decision-table model and a combinatorial reduction from its own propositional formulas. It did not formalise the complexity classes, which need a model of computation, polynomial-time bounds and encodings.

This paper supplies them. We compose latc-check's model with Complexitylib [6], a Lean 4 library of complexity theory on multi-tape Turing machines with a machine-checked Cook–Levin theorem, and prove the classification end to end, together with results that neither earlier development contains. On paper the classification is elementary: with no rows, completeness is unsatisfiability of the constraints. The contribution is its mechanisation on Turing machines with an adequacy theorem, the tractability boundary with an explicit degree, and the one-call Valiant theorem. Every theorem we cite depends only on Lean's three standard axioms, and the build contains no `sorry`.

Scope. Throughout, a row of the table is a complete tuple of condition values, as in latc-check. Tables with compact rows, such as DMN rules with wildcards or ranges, are a different problem: there completeness is coNP-hard even without constraints (Section 8).

Informal claims. A few claims are argued on paper only and are not theorems of the development. They are marked with a dagger, as in "P†", both in the text and in Table 2. Every other result is a Lean theorem, cited by its name.

Contributions. Table 1 separates what this paper adds from what it reuses.

1. **The classification, machine-checked end to end** (Sections 4 and 5): non-emptiness of $M(T_0)$ is NP-complete, completeness coNP-complete and counting #P-complete, for finite domains with clause constraints, with every number of the input written in binary. Hardness holds already for instances with Boolean domains and empty tables: a Lean theorem shows that the reduction produces only such instances (Section 4).
2. **Adequacy** (Section 3): the encoded problems are exactly latc-check's set-based ones.
3. **One-call hardness for monotone two-literal counting** (Section 6): a definition of one-call reductions in Complexitylib's model, and a new formal proof in that model of a one-call version of Valiant's theorem, by classical techniques. From it,

Table 1 What is reused and what is new.

result	before this paper	here
decision-table model, missing set	latc-check (Lean)	reused
SAT reduction to missing states	latc-check, from its own formulas	new, from Complexitylib's CNF
NP/coNP-completeness, #P-completeness	on paper [4, Prop. 5]	Turing machines, in Lean
adequacy of the encoded problems	–	new
polynomial cases: no constraints, fixed k	analogues for firewalls [7] and for unknown entries [8]	new for decision tables
explicit degree in the number of attributes	–	new
one-call reductions, one-call Valiant	Valiant (Turing reductions); Senellart–Gnatenko (one call, logical model, Lean library)	new formal proof in Complexitylib's model

counting the missing combinations stays #P-complete under one-call reductions when every constraint is $A_i = 1 \vee A_j = 1$, for tables with arbitrary rows (hard already with none).

4. **The tractability boundary** (Section 7): polynomial time without constraints, and with a fixed number of attributes, for both decision problems and for counting, with an explicit degree linear in the number of attributes.
5. **An account of the engineering** (Section 10): the size of each part, what Complexitylib provided and lacked, how the polynomial-time proofs were automated, how the instance-dependent bounds were kept explicit, the lemmas that could be reused, and how an AI system was used.

Section 2 recalls Complexitylib and latc-check, Section 3 fixes the input language, Sections 4–7 prove the results, Section 8 discusses related work, Section 9 the trusted base and Section 10 the engineering of the development. Appendix A lists the main Lean statements, Appendix B the trusted definitions and Appendix C the artifact.

2 Preliminaries

Machines, encodings and classes. Complexitylib, built on CSLib [9] and Mathlib, works with multi-tape Turing machines over a binary alphabet. A type α is given an encoding, an injection $\alpha \hookrightarrow \{0,1\}^*$; pairs, lists and natural numbers have standard encodings, numbers in binary (`encNat`) or in unary (`encUnary`). A function f is computable in polynomial time between two encodings when some machine maps the encoding of every a to the encoding of $f(a)$ within a polynomial in the input length. A language is in P when its characteristic function is; in NP when it is the set of inputs having a certificate of polynomially bounded length accepted by a polynomial-time verifier; and coNP is the class of complements. A function f into

$\mathbb{N}$ is in #P [10] when $f(a)$ is the number of certificates of length exactly $p(|a|)$, for a polynomial p, that a polynomial-time verifier accepts on a. A library lemma, `mem_SharpP_of_polyTime_length`, lets the certificate length be any function of the input that is computable in polynomial time with its value written in unary; our counting verifiers use it.

Reductions. Three notions are used. A *many-one* reduction from A to B is a polynomial-time r with $a \in A \iff r(a) \in B$; NP-, coNP-hardness and completeness are defined with it. A *parsimonious* reduction from f to g is a polynomial-time r with $f(a) = g(r(a))$; Complexitylib defines #P-hardness with it. These two are Complexitylib's. The third is defined in this development; it is the polynomial-time *metric reduction* that Krentel defines between functions into $\mathbb{N}$ for optimisation problems [11]:

Definition 1 (one-call reduction) f reduces to g with one call if there are polynomial-time functions r and *post* with $f(a) = \mathit{post}(a, g(r(a)))$ for every a, where *post* reads a and the answer $g(r(a))$ written in binary. g is *one-call #P-hard* if every function in #P, under any encoding, reduces to it with one call, and *one-call #P-complete* if it is moreover in #P.

The running time of *post* is polynomial in the length of a and of the oracle's answer. For a target g whose values have polynomially many bits, as for every function in #P, a one-call reduction is a polynomial-time Turing reduction making exactly one oracle query, whose answer is post-processed together with the input, and one-call hardness implies hardness under polynomial-time Turing reductions[†], the notion of Valiant [12]. (Without that bound the notion would be too weak: a g with doubly exponential values gives *post* exponential time in $|a|$ while carrying no information. We use one-call hardness only for #Monotone-2SAT, whose value is at most $2^{\#\text{variables}}$, and for a target proved to be in #P.) Parsimonious reductions are one-call, with $\mathit{post}(a, y) = y$ (`ParsimoniousReducible.oneCallReducible`); so #P-hardness implies one-call #P-hardness (`SharpPHard.oneCallSharpPHard`), and a parsimonious reduction followed by a one-call reduction is one-call (`ParsimoniousReducible.trans_oneCall`). Zankó defines #P-completeness with a variant of many-one reductions and shows the $(0, 1)$-permanent complete under it [13]. One-call hardness is weaker than parsimonious hardness, which matters below: a problem whose decision version is trivial cannot be parsimoniously #P-hard unless P = NP, and can still be one-call #P-hard. Our hardness proofs start from Complexitylib's Cook–Levin theorem and its parsimonious form: SAT is NP-hard, UNSAT coNP-complete, and #SAT #P-hard under parsimonious reductions.

latc-check's model. Attributes are indexed by a finite type ι, attribute i having the declared finite domain D_i, so the space of condition tuples $\mathcal{D} = \prod_i D_i$ is finite. Constraints are a decidable predicate C on tuples, and a table is represented by its set T_0 of condition tuples. The valid space is $\mathcal{V} = \{\tau \in \mathcal{D} : C(\tau)\}$, the missing set is $M(T_0) = \mathcal{V} \setminus T_0$, and the table is complete when $M(T_0) = \emptyset$. In Lean these are `validSpace C` and `missing C T`, both `Finset`s, so duplicate rows play no role. The present development reuses this model; it does not use latc-check's own reduction. Two

lemmas check that latc-check's clause constraints and satisfiability agree with Complexitylib's CNF formulas (`holdsAll_eq_eval`, `satisfiable_decode_iff`); the hardness proofs below use a new reduction stated directly on Complexitylib's CNF.

3 Instances, encoding and adequacy

Instances. An instance is a triple $I = (d, S, T)$ of lists of natural numbers:

- $d = [d_0, \ldots, d_{n-1}]$, the domain sizes; attribute i ranges over $\{0, \ldots, d_i - 1\}$, and n, the number of attributes, is the length of d;
- S, the clause constraints: a list of clauses, each a list of conditions (i, v), read $A_i = v$;
- T, the rows of T_0: a list of lists of values.

A *state* is a list s of length n with $s_i < d_i$. It satisfies the condition (i, v) when $i < n$ and $s_i = v$, and it is *valid* when every clause of S has a condition it satisfies. Every triple is an instance; nothing is rejected as malformed, and the semantics settles each degenerate case. A condition on a nonexistent attribute, or with a value outside the domain, is never satisfied; an empty clause makes every state invalid; a domain of size zero leaves no states; a row of the wrong length or with an out-of-domain value covers no state; and repeated rows or conditions change nothing. Then

$$\begin{aligned} \mathit{MissingNonempty} &= \{I : \text{some valid state is not a row of } T\}, \\ \mathit{Complete} &= \overline{\mathit{MissingNonempty}}, \end{aligned}$$

and $\mathit{missingCount}(I)$ is the number of valid states that are not rows.

A running example. Take two attributes with $d = [3, 2]$, the constraint $A_0 = 0 \vee A_1 = 1$, and the rows $(0, 0)$, $(0, 1)$, $(1, 1)$:

$$I = \big([3, 2],\ [[(0, 0), (1, 1)]],\ [[0, 0], [0, 1], [1, 1]]\big).$$

Of the six states, four are valid: $(0, 0)$, $(0, 1)$, $(1, 1)$ and $(2, 1)$. The first three are rows, so $M(T_0) = \{(2, 1)\}$: the instance is in *MissingNonempty*, it is incomplete, and $\mathit{missingCount}(I) = 1$.

Encoding and input size. The main encoding, `encInstBin`, writes the triple with Complexitylib's encodings of pairs and lists and every number in binary: the domain sizes, the attribute index and value of each condition, and the values of each row. The number of attributes is not written separately; it is the length of the list d. The input size $|x|$ in every polynomial bound below is the length of this encoding of the instance x. With binary domain sizes, $|\mathcal{D}| = \prod_i d_i$ can be exponential in $|x|$ even for one attribute.

Checking a condition (i, v) against a state reads s_i at a binary index i. We use Complexitylib's `listGetD_binary`, which walks the list while reading the bits of the index and returns a default when the list ends, so the cost is polynomial in the lengths of the list and of the index's binary encoding; the index is never expanded to unary. The values are shifted by one before the lookup, so that the default 0 matches no value: an out-of-range attribute fails the condition, as the semantics requires. A second

encoding, `encInst`, writes the attribute index in unary. The identity is polynomial-time from `encInst` to `encInstBin` (`encInst_to_encInstBin_polyTime`), so hardness for the unary encoding transfers to the binary one. The NP-, coNP- and #P-completeness results of Sections 4–6 are proved for both encodings, and the polynomial-time results of Section 7 for `encInstBin`, each with a `_unary` form for `encInst` (Appendix A). The hardness results on the Boolean subclass (Section 4) and the explicit-degree bounds of Section 7 are stated for `encInstBin` only.

Adequacy. An instance denotes a table in latc-check's model: attributes `Fin n`, domains `Fin d`$_i$, the constraint predicate given by the clauses, and as T_0 the tuples whose list of values is a row. The word problems are exactly latc-check's:

```
theorem mem_MissingNonempty_iff (I : Inst) :
    I ∈ MissingNonempty ↔ (missing I.con I.table).Nonempty
theorem mem_Complete_iff (I : Inst) : I ∈ Complete ↔ missing I.con I.table
    = ∅
```

A third theorem, `coe_missing_map_toList`, shows that the tuples of $M(T_0)$, read as lists, are exactly the witnesses of *MissingNonempty*, and *missingCount* is defined as the cardinality of latc-check's missing set. Every complexity result below is therefore a statement about latc-check's `missing` on clause-constrained tables; it says nothing about constraint languages other than clauses.

4 The decision problems

Theorem 2 *Under* ***`encInstBin`****, MissingNonempty is NP-complete and Complete is coNP-complete.*

Proof sketch Membership. The certificate is a missing state. The verifier checks that it lies in the domains, satisfies every clause and is not a row; it runs in polynomial time (`checkBin_polyTime`). A state in the domains has $s_i < d_i$ for every i, so its encoding is no longer than that of d, which bounds the certificate length by $|x|$. *Hardness.* Given a CNF formula φ, renumber its variables densely as $0, \ldots, n-1$ and build the instance with n attributes of domain $\{0,1\}$, one clause per clause of φ (the literal x_i becomes $A_i = 1$, and $\neg x_i$ becomes $A_i = 0$), and no rows. Its valid states are the satisfying assignments of φ, and with no rows they are all missing. So φ is satisfiable iff the instance is in *MissingNonempty*, and unsatisfiable iff it is complete: a reduction from SAT and from UNSAT, polynomial-time (`General.fromSAT_polyTime`). Composed with Complexitylib's Cook–Levin theorems and with the identity re-encoding, this gives the theorem (`MissingNonempty_npComplete_bin` , `Complete_coNPComplete_bin`). By adequacy the same holds for $\{I : M(T_0) \neq \emptyset\}$ and $\{I : M(T_0) = \emptyset\}$ stated with latc-check's `missing`. □

Hardness on Boolean domains with empty tables. The reduction produces only instances whose domains are all $\{0,1\}$ and whose table is empty: writing *BoolEmpty* for the class of such instances, `fromSAT_mem_boolEmpty` states that every *fromSAT*(φ) is in it. Hardness therefore holds already on that class, and Lean states it under `encInstBin` , with the convention of Section 6 that a restricted problem contains its restriction:

MissingNonempty ∩ *BoolEmpty* is NP-hard (`MissingNonemptyBoolEmpty_npHard_bin`) and *Complete* ∩ *BoolEmpty* coNP-hard (`CompleteBoolEmpty_coNPHard_bin`). The promise version, which asks to tell the instances of *BoolEmpty* with $M(T_0) \neq \emptyset$ from those with $M(T_0) = \emptyset$ and allows any answer elsewhere, is NP-hard as a promise problem in Complexitylib's sense (`missingNonemptyPromise_npHard_bin`). The counting reduction of Theorem 3 is the same map, and gives the same for counting (`missingCountBoolEmpty_sharpPHard_bin`).

The empty table shows that the hardness lies in the constraints: with $T_0 = \emptyset$, completeness is unsatisfiability of the constraints. Rows that are complete tuples only remove single states, and Section 7 shows that without constraints they cannot make the problems hard; compact rows could (Section 8).

5 Counting

Theorem 3 *Under* `encInstBin`*, missingCount is #P-complete under parsimonious reductions.*

Proof sketch Membership. Membership needs every missing state to have exactly one accepted certificate. Let L be the length of the encoding of the domain sizes d. A certificate is a bit string of length L. The verifier decodes the first n numbers from it into a state s and accepts when s is missing *and* the certificate equals the canonical certificate of s: the encoding of s followed by zeros up to length L. The encoding of a state in the domains is no longer than that of d, so every missing state has a canonical certificate of length L; the canonical certificate determines the state; and any other string decoding to s is rejected. So the accepted certificates of length L are in bijection with $M(T_0)$ (`missingCount_eq_acceptCount`), and both the verifier and L are polynomial-time. Without the canonicity test, a state could have several accepted certificates (different padding) and the count would be wrong.

Hardness. The reduction of Theorem 2 is parsimonious: the satisfying assignments of φ and the missing states of its instance are in bijection, so $\#\mathrm{SAT}(\varphi) = \mathit{missingCount}(\mathit{fromSAT}(\varphi))$ (`sharpSAT_eq_missingCount`). Composition with Complexitylib's parsimonious Cook–Levin theorem gives `missingCount_sharpPComplete_bin`. □

6 Monotone two-literal constraints and Valiant's theorem

Russell [4] notes that counting the valid states stays #P-complete when every clause is $A_i = 1 \vee A_j = 1$ over domains $\{0, 1\}$, citing Valiant [12]. Deciding such instances is easy: with domains $\{0, 1\}$ and only positive two-literal clauses on existing attributes, the all-ones state is valid; the shape tests also require every condition to name an existing attribute, so every such instance with no rows has a missing state (`mem_missingNonempty_of_pos2B`). With rows, deciding is still polynomial[†]: a partial assignment to a positive two-literal CNF extends to a model if and only if no clause has both literals set to 0 (set the rest to 1), so backtracking lists the valid states with polynomial delay, and listing $|T| + 1$ of them decides whether one is missing. So neither count can be #P-hard under parsimonious reductions unless P = NP[†], and the right notion is the one-call reduction of Definition 1.

Restricted problems. A problem restricted to a class K of instances is stated in this development as a problem on all instances that contains its restriction. A restricted decision problem is the set $\{I : I \in K \wedge I \in A\}$, so that its decider also checks $I \in K$, and a restricted count returns 0 outside K, as Complexitylib's #3SAT does. The counts of this section follow the convention, and so do the restricted sets of Sections 4 and 7. It is harmless because every class used is decidable in polynomial time: the shape tests below (`pos2B_polyTime_bin`, `pos2AnyB_polyTime_bin`), *BoolEmpty*, "no constraints" and "at most k attributes", the last three plain comparisons on the input. The *promise version* of a restricted problem asks for the right answer only on the instances of K. A polynomial-time decider for the restricted set solves the promise version, so the upper bounds of Section 7 hold for both forms; conversely, a solver for the promise version, run after the test for K, decides the restricted set†. The hardness proofs map their inputs into K, so they show the promise versions hard as well. Lean states promise versions in two places: `missingNonemptyPromise_npHard_bin` (Section 4), and Statement 2 of the explicit degree (Section 7), whose input type is the subtype of instances with at most k attributes. For the counts below, the promise version is the count on the instances that pass the shape test; the reduction leaves that class only on formulas where both sides are 0 by the convention, and the promise versions of the counts are not stated separately in Lean†.

Let *SharpMonotone2SAT* be the number of models of a CNF formula whose clauses are all $x_i \vee x_j$, and *countValidMonotone2* the number of valid states of an instance with domains $\{0, 1\}$, clauses of two conditions with value 1 on existing attributes, and no rows. Let *countMissingMonotone2* be the same without the condition on rows: the number of missing combinations $|M(T_0)|$ of an instance with those domains and constraints and any rows (`countMissingMonotone2_eq_card_missing`). As with Complexitylib's #3SAT, the three functions are total and return 0 on inputs of any other shape. Both counts are in #P, and #Monotone-2SAT reduces to each parsimoniously (`sharpMonotone2SAT_reduces`, and `sharpMonotone2SAT_reduces_countMissing` in `Latc.PaperD.Valiant`): the reduction produces either a monotone instance with no rows, where the two counts agree, or a fixed instance of the wrong shape, where both are 0. What remains is the hardness of #Monotone-2SAT itself.

Theorem 4 (one-call Valiant) *#Monotone-2SAT is one-call #P-hard. Hence countValidMonotone2 and countMissingMonotone2 are one-call #P-complete under* ***`encInstBin`*** *and* ***`encInst`****.*

Valiant proved #Monotone-2SAT #P-complete under polynomial-time Turing reductions [12]. Its relation $x \vee y$ is not affine, so it also lies on the hard side of Creignou and Hermann's dichotomy for counting generalised satisfiability [14]. Senellart and Gnatenko's Lean library [15] machine-checks a one-call form in its descriptive-complexity setting, by a chain of four reductions: from #1-in-SAT to counting cliques, to counting independent sets of a given size, to counting all independent sets with one call, by attaching pendant leaves to every vertex and reading one digit of the answer, and to #Monotone-2SAT. Our proof uses the same two devices, a parsimonious encoding of models as independent sets of a fixed size, and pendant-leaf

amplification followed by digit extraction. It differs from theirs in fusing the chain into one reduction from #SAT, through the "first true literal" graph below, which handles any CNF and needs no counting problem on graphs. What it adds is a new formal proof in Complexitylib's model, where every step is a polynomial-time Turing-machine computation; it claims no new mathematics. Since #SAT is #P-hard under parsimonious reductions, by Definition 1 it suffices to reduce #SAT to #Monotone-2SAT with one call.

Step 1: models as independent sets. Let φ have variables $0, \ldots, n-1$ (after dense renumbering) and clauses $c_1, \ldots, c_m$. Build a graph G with a vertex (i, b) for each variable i and value b, and a vertex o for each literal occurrence, read "o is the first true literal of its clause". Edges join $(i, 0)$ and $(i, 1)$; any two occurrences of the same clause; an occurrence and the value of its variable that falsifies it; and an occurrence and each value that makes an *earlier* literal of its clause true. The vertices fall into $K = n + m$ cliques, $\{(i, 0), (i, 1)\}$ for each variable and the occurrences of each clause, so every independent set has at most K vertices. An independent set of exactly K vertices picks a value for each variable, which is an assignment σ, and an occurrence in each clause that σ makes true, with every earlier literal false. That occurrence is unique given σ, so these independent sets are in bijection with the models of φ (`modelCount_eq_card`). The "first true literal" choice is what makes the correspondence exact, for any CNF with densely numbered variables (the hypotheses of `modelCount_eq_card`): repeated and complementary literals need no special case. An empty clause has no vertex, so no independent set reaches K, and the count is 0, as it should be.

For example, $\varphi = x_0 \vee x_1$ has $n = 2$, $m = 1$, $K = 3$, vertices $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$, o_1 for x_0, o_2 for x_1. The edges at o_1 go to o_2 and $(0, 0)$; those at o_2 go to o_1, $(1, 0)$ and $(0, 1)$. The independent sets of size 3 are $\{(0, 1), (1, 0), o_1\}$, $\{(0, 1), (1, 1), o_1\}$ and $\{(0, 0), (1, 1), o_2\}$: three, one per model.

Step 2: a monotone formula. Let ν be the number of vertices of G, and give every vertex $t = n+1$ fresh pendant leaves. The monotone formula ψ has a variable per vertex and leaf, and a clause $x_u \vee x_w$ per edge $\{u, w\}$ of the graph with leaves. Its models are the vertex covers, the complements of the independent sets. An independent set of the graph with leaves consists of an independent set J of G together with any subset of the leaves of the $\nu - |J|$ vertices outside J, so

$$y \;=\; \#\mathrm{SAT}(\psi) \;=\; \sum_{J \text{ independent in } G} 2^{\,t\,(\nu - |J|)} \qquad (\texttt{modelCount_psi}).$$

Every vertex has leaves, so every variable of ψ occurs in some clause, and ψ has exactly two positive literals per clause.

Step 3: one digit. Write y in base 2^t. Multiply by 2^{tK} and divide by $2^{t\nu}$:

$$\left\lfloor \frac{y \cdot 2^{tK}}{2^{t\nu}} \right\rfloor = \left\lfloor \sum_{J} 2^{\,t\,(K - |J|)} \right\rfloor.$$

Since $|J| \leq K$, every exponent is a nonnegative multiple of t, so the sum is an integer and the floor is exact. Sets with $|J| < K$ contribute multiples of 2^t, and sets with $|J| = K$ contribute 1 each. Their number is $\#\mathrm{SAT}(\varphi) \leq 2^n < 2^t$. So

$$\#\mathrm{SAT}(\varphi) \;=\; \left\lfloor \frac{y \cdot 2^{tK}}{2^{t\nu}} \right\rfloor \bmod 2^t \qquad (\texttt{sharpSAT_eq_post}).$$

Multiplying before dividing avoids computing $\nu - K$, which can be negative when φ has empty clauses (K counts them, ν does not). Then every term $2^{t(K-|J|)}$ has $|J| < K$, so the quotient is a multiple of 2^t and the digit is 0, which is correct. The post-processing needs only binary multiplication, division, remainder and powers of two on numbers of polynomially many bits, all polynomial-time in Complexitylib; the construction of ψ is polynomial-time; and the oracle is called once, on ψ (`toMonotone_polyTime`, `post_polyTime`, `sharpSAT_oneCallReducible`). Composing with the parsimonious #P-hardness of #SAT gives `sharpMonotone2SAT_oneCallSharpPHard`, and with the reductions to *countValidMonotone2* and *countMissingMonotone2*, the second half of Theorem 4 (Appendix A). The development also keeps the conditional form, `countValidMonotone2_oneCallSharpPComplete_bin`, which takes #Monotone-2SAT's hardness as a hypothesis; Theorem 4 discharges it.

7 Where the hardness comes from

The reductions of Sections 4 and 5 use clause constraints and as many attributes as the formula has variables. Removing either ingredient makes all three problems polynomial-time (Table 2). As in Section 6, each restricted problem below contains its restriction.

No constraints. Then every state is valid and $|M(T_0)| = \prod_i d_i - r$, where r is the number of *distinct* rows that are states (`missingCount_eq_of_noConstraints`). The product can be exponential in $|x|$, but its binary encoding is no longer than those of the factors together, so a fold that multiplies in binary stays polynomial: each intermediate product has at most $|x|$ bits. The count is therefore polynomial-time, and so are completeness and non-emptiness, which compare $\prod_i d_i$ with r (`missingCountNoConstraints_polyTime`, `completeNoConstraints_mem_P`, `missingNonemptyNoConstraints_mem_P`).

At most k attributes: deciding. With binary domain sizes, $\mathcal{D}$ cannot be enumerated even for $k = 1$. Let N be the number of value entries written in the instance, in conditions and in rows, and call a number *unwritten* if it occurs in none of them. An unwritten value satisfies no condition, and a state containing one is not a row; so in a missing state, an unwritten value can be replaced by any other unwritten value of the same domain, and the state stays missing. At most N of the numbers $0, \ldots, N$ are written, so one of them, u, is unwritten. If a missing state uses an unwritten value $v > N$ in attribute i, then $d_i > v > u$, so u is in the domain and can replace v. Hence if a missing state exists, one exists whose values lie among $0, \ldots, N$ and the written values: at most $2N + 1$ candidates per attribute and $(2N + 1)^k$

candidate states (`length_candidateStates_le`, `exists_missing_candidate_iff`). Checking each candidate is polynomial, so both decision problems are in P for each k (`missingNonemptyFixedAttrs_mem_P`, `completeFixedAttrs_mem_P`).

At most k attributes: counting. A candidate state stands for all the states obtained by replacing its unwritten values, so counting must weigh it. For each attribute, split its domain into classes: each written value inside the domain alone, and one class holding all the unwritten values of the domain (possibly empty), represented by u. Which conditions a state satisfies, and whether it is a row, depend only on its vector of classes, so

$$|M(T_0)| = \sum_{\substack{\gamma \text{ a class vector,} \\ \text{valid and not a row}}} \prod_i |\gamma_i| \qquad (\texttt{missingCount_eq_weighted_of_le}),$$

over at most $(N+1)^k$ class vectors (`length_classStates_le`). Each product is at most $\prod_i d_i$, so the sum is computed in binary by the bounded-growth fold of the no-constraint case, and the count is polynomial-time for each k (`missingCountFixedAttrs_polyTime`).

An explicit degree. "Polynomial time for each k" allows the polynomial to depend on k arbitrarily. We bound its form: the exponent is linear in the number of attributes, with constants that are proved to exist and are not computed. The constants are written c, e, e_0 and e_1 below; the Lean statements call the exponent constants `d`, $\texttt{d}_0$ and $\texttt{d}_1$. Two names of bound frameworks occur. The statements use `ComputableTidilyInTimeAndSpace f encIn encOut t s` (from CSLib), which says that one machine computes f on every input a within exactly the time $t(a)$ and the space $s(a)$. The proofs use Complexitylib's `ComputableTidilyInO f encIn encOut T S`, which is defined from it: for some constant c, the bounds $c\,(T(a)+1)$ and $c\,(S(a)+1)$ hold and the output has length at most $c\,(S(a)+1)$. The second form keeps explicit bound functions and a single constant through composition, case splits and folds; the lemma `timeAndSpace_of_inO_xp` converts it to the first at the end (Section 10). The following are proved, each for non-emptiness, completeness and counting, each for time and space, under `encInstBin`:

1. *One machine for all instances.* There are c, e such that one machine decides non-emptiness on every instance within $c\,(|x|+1)^{en+e}$, where n is the instance's number of attributes (`missingNonempty_explicitDegree`; Appendix A).
2. *Uniform constants in k* (`missingNonemptyFixedAttrs_uniform`). There are c, e_0, e_1 such that for every k some machine decides the problem within $c\,(|x|+1)^{e_0 k+e_1}$ on every instance with at most k attributes. Its inputs are the instances with at most k attributes, encoded by `encInstBin`: the Lean input type is the subtype `{I : Inst // I.n ≤ k}`, so nothing is claimed about other inputs, and the statement is a promise version in the sense of Section 6. The proof takes the machine of 1 for every k; the statement asserts only that a machine exists for each k.
3. *k in the input.* There are c, e such that one machine, given k in unary and an instance, decides "at most k attributes and $M(T_0) \neq \emptyset$" within $c\,(|(k,x)|+1)^{ek+e}$ (`missingNonemptyWithBound_explicitDegree`).

Table 2 Where the hardness comes from. A dagger marks a claim argued on paper only; every other entry is a Lean theorem. The hardness of the first row holds already on Boolean domains with $T_0 = \emptyset$.

rows	constraints	attributes	deciding	counting	hardness via
tuples	clauses	unbounded	NP-, coNP-complete	#P-complete	many-one, parsimonious
tuples	none	unbounded	P	polynomial-time	–
tuples	clauses	at most k	P, degree $O(k)$	polynomial-time, degree $O(k)$	–
tuples	monotone two-literal	unbounded	P† (trivial if $T_0 = \emptyset$)	one-call #P-complete	one-call
compact (wild-cards)	none	unbounded	coNP-complete†	#P-complete†	DNF tautology, #SAT

The degree grows linearly with the number of attributes. This is an XP bound, $|x|^{O(k)}$; fixed-parameter tractability would need $f(k) \cdot |x|^{O(1)}$, and it is unlikely†. With k attributes, non-emptiness is a constraint satisfaction problem with k variables, and such problems are W[1]-hard with the number of variables as parameter: Papadimitriou and Yannakakis prove it for conjunctive queries, by a reduction from Clique [16], a result already stated as Theorem 1 of its conference version (PODS 1997). An argument we have not formalised reduces Multicolored Clique, which is W[1]-complete [17, Theorem 13.25], to non-emptiness with k attributes directly†. Give attribute i the vertices of colour class i as values, no rows, and for each non-adjacent pair u, w in classes $i \neq j$ the clause $\bigvee_{v \neq u} A_i = v \vee \bigvee_{v \neq w} A_j = v$ ("not both $A_i = u$ and $A_j = w$"); the valid states are exactly the multicoloured cliques.

What is not proved. We do not prove that, for each fixed k, a separate machine with k built in, which rejects instances with more than k attributes, meets the bound with constants independent of k. Testing $n \leq k$ inside the machine goes through Complexitylib's case-split and constant-output lemmas, whose constants depend on the machine, and making them uniform would need changes to the library. Statements 2 and 3 cover the same ground: the first on the instances where the answer can be positive, the second on all instances with k given as input.

8 Related work

Compact rows. Our rows are complete tuples, as in latc-check. In DMN and in many rule bases a row is compact: a conjunction of tests over the declared finite domains, with wildcards or ranges, standing for many tuples. Then the rows form a DNF formula and the uncovered inputs are the models of its negation, a CNF. Both directions hold†: any CNF φ arises this way, from the wildcard rows given by the terms of $\neg\varphi$,

with the uncovered tuples exactly the models of φ. So, even without constraints, completeness is the DNF tautology problem, coNP-complete, and counting the uncovered inputs is parsimoniously equivalent to #SAT, hence #P-complete by the parsimonious Cook–Levin theorem; tautology of DNF is coNP-complete by Cook's theorem [18]; membership holds because an uncovered tuple is a certificate of polynomial size, even with domain bounds written in binary, checked against each row in polynomial time. Russell [4] makes the same observation. Elmallah and Gouda's firewall rules are compact rows of this kind: each rule tests every field against an interval of integers [19]. Thiemann and Yamada's coNP-hardness reduction turns each clause into a row of true, false and wildcard patterns, so it already shows coNP-hardness for decision tables with wildcard rows [20]; what this paper adds is the case of rows that are complete tuples, constraints, counting and the mechanisation. With complete tuples as rows, as here, the hardness comes from the constraints (Section 7).

Hardness of coverage. Hardness of rule coverage is not new in itself. Elmallah and Gouda prove firewall completeness NP-hard, by a reduction from a slice-probing problem that they reduce from 3-SAT [19]. The reduction maps to the complement of completeness, so by its form it shows coNP-hardness; for membership they only remark that some of their problems can be shown to be in NP and the others in coNP. Our account follows their 2013 technical report of the same title. Thiemann and Yamada prove pattern completeness coNP-hard and verify their decision procedure in Isabelle, the hardness proof being on paper [20]. According to its abstract, the paper of Kapur et al. shows sufficient completeness coNP-complete for term rewriting systems with free constructors, and state that their complexity results also apply to ground reducibility [21]; Thiemann and Yamada cite the paper for the coNP-completeness of quasi-reducibility. Deciding whether a point lies outside a union of boxes is NP-hard [22], and computing the volume of the union is #P-hard [23]. For firewalls with a fixed number of header fields, Hamilton, Roughan and Nguyen give polynomial-time algorithms [7], an analogue of our fixed-k results.

Decision tables and tabular specifications. Checking tables for completeness and consistency is long established: in expert-system checkers [2], for decision tables [24], for SCR and RSML requirements [25, 26], and for DMN, where Calvanese et al. give analysis algorithms [3, 27] and, with background knowledge, an ExpTime upper bound [28]. PVS generates coverage and disjointness obligations for tabular specifications [29]; these are proof obligations about given tables, and come with no complexity results. Menghi et al. prove completeness of Simulink Requirements Tables undecidable, by reduction from Hilbert's tenth problem, because their conditions are arithmetic over infinite domains, and check it with SMT [30, 31]. Vu and Demetrovics study decision tables whose entries may be unknown: for one given input, deciding its possible and certain decisions is NP- and coNP-complete, and counting the valuations yielding a decision #P-hard, all by encoding a formula in the constraints on the unknowns; a fixed number of unknowns is polynomial, another analogue of our fixed-k case [8]. They define completeness for a single input but state no complexity for deciding it. Their model asks what a partly unknown table says about one input; ours asks whether a fully specified table covers every input its declared domains and constraints allow.

Rule-base verification. Levy and Rousset gave the first complexity results for verifying rule bases [32, 33]: whether no legal input derives a bad output, for relational inputs and Horn rules, NP-complete to undecidable depending on recursion, and Π_2^p-complete with order predicates and negation. Their "input completeness" constraints restrict which inputs are legal, and they leave output completeness open. Coverage of a propositional table could be phrased in their framework, but it lies in coNP $\subseteq \Pi_2^p$: coNP-complete for completeness, as shown here, with NP for non-emptiness and #P for counting.

Apart from the argument on paper in [4], and to our knowledge, none of these works classifies the completeness of decision tables with complete tuples as rows over declared finite domains, or counts their missing combinations, and none of the hardness proofs is machine-checked.

Mechanised computability and complexity. Cook–Levin has been mechanised in ACL2 [34], Coq [35] and Isabelle [36]; reductions between Karp's problems are verified in Isabelle [37]; and Senellart and Gnatenko prove all 21 of Karp's problems NP-complete in Lean [38], with NP defined in descriptive-complexity terms and first-order reductions; their library also proves #Monotone-2SAT one-call #P-complete in that setting [15]. Mathlib [39] itself defines polynomial-time computability for its stack-based Turing machines: `TM2ComputableInPolyTime`, in the file `Mathlib/Computability/TuringMachine/Computable.lean` by Spelier and van Gent [40]. Turing machines and computability have been mechanised in Isabelle/HOL [41] and Coq [42], computability through partial recursive functions in Lean [43], and time complexity through the weak call-by-value λ-calculus in Coq [44]. Our setting needs explicit encodings and running times; in exchange its classes are the textbook ones on Turing machines, so polynomial time, binary encodings of numbers and the size of the input, the quantities the tractability results of Section 7 are about, are first-class.

9 Trusted base and audit

Trust. The axiom audit prints the axioms of 328 of the development's 345 theorem and lemma declarations, among them every theorem this paper cites (the other 17 are helper lemmas used by audited theorems); each uses only `propext`, `Classical.choice` and `Quot.sound`, or fewer, and none uses `sorryAx`. This shows that no proof is left unfinished and no axiom is added. It does not show that the definitions are the intended ones. The trusted base is Lean's kernel (we did not replay the proofs in an external checker); Complexitylib's definitions of machines, running time and space (`ComputableTidilyInTimeAndSpace`, which carries the explicit-degree claims), encodings, the classes P, NP, coNP and #P, the reductions, promise problems and the notions of completeness built on them, and its CNF semantics and #SAT; our encodings of instances; Definition 1; and the problem definitions themselves: *MissingNonempty* with `InDomains`, `CondHolds` and `Valid`, *missingCount*, the monotone shape tests, *SharpMonotone2SAT* and *BoolEmpty*. Appendix B copies every one of our trusted definitions verbatim from the source. The adequacy theorems shrink the list: the general problems are proved equal to latc-check's, so for them a reader need trust only latc-check's

definition of the missing set and our map from an instance to the table it denotes (`Inst.con`, `Inst.table`), each a few lines (Sections 2 and 3). For the monotone counts, adequacy holds only on instances that pass the shape test, so the shape test is trusted; an earlier version of it accepted conditions on nonexistent attributes, which the present one rejects.

Worst case and practice. The classification rules out a polynomial algorithm for all tables with clause constraints, unless P = NP; it does not say that tables met in practice are slow to check. For tables whose rows are complete tuples, few attributes or the absence of constraints give the polynomial regimes of Section 7, and the difficulty comes from constraints across many attributes; neither the size of the domains nor that of the table makes the problems hard. For compact rows, as in DMN, the rows themselves can carry the hardness (Section 8), and our tractability results do not apply.

10 Engineering the development

This section reports what the formalisation took and what it taught. Its figures come from the repository at the commit described in Appendix C and from the commit history, which is summarised in the repository's README. No working hours were recorded, so it reports lines, commits and dates only.

10.1 Size and effort per part

Table 3 lists the modules. The development has 4 520 lines of Lean in 15 files, besides the two axiom-audit files (383 lines), with 345 theorem and lemma declarations; it sits on latc-check's model, copied unchanged into `LatcFormal/`. By lines, the effort fell unevenly on four parts:

- *Encodings and the decision problems*, 411 lines (`General.lean` with the unary index, `Binary.lean` with the binary one). Most of it is the certificate check and its polynomial time; the binary index cost 206 lines, mainly the re-encoding and the shifted lookup of Section 3.
- *Adequacy*, 152 lines (`Adequacy.lean`): the map from a state to a tuple of `Fin` values, its injectivity and the two adequacy theorems.
- *The explicit degree*, 692 lines (`ExplicitDegree.lean`), for a statement that is a corollary on paper: almost as much as the 825 lines of encodings, adequacy and counting together (`General`, `Binary`, `Adequacy`, `CountingGeneral`). The cost is bookkeeping: every composition step must carry an explicit bound (Section 10.4).
- *Valiant's theorem*, 1 140 lines in four files, the largest part; 518 of them (`Valiant/Counting.lean`) prove the bijection between models and independent sets of size K.

The tractable side (`Tractable.lean`, 1 099 lines) is the largest single file: the enumeration of candidate states needs an induction on k for its polynomial time, and the counting with classes adds the partition of each domain. Its first Lean commit (28 October 2026) holds the Boolean model and the general NP- and coNP-completeness;

Table 3 The modules of `LatcComplexity/`: lines, theorem and lemma declarations, and definitions (`def`, `abbrev`, `instance`). The two audit files are not counted in the total.

file	lines	thms	defs	contents
`Bridge.lean`	160	16	11	Boolean model; agreement with latc-check's clauses
`General.lean`	205	15	13	instances, `encInst`, NP- and coNP-completeness
`Adequacy.lean`	152	13	6	the table an instance denotes; adequacy
`Counting.lean`	140	10	2	Boolean model: counting in #P
`CountingGeneral.lean`	262	20	7	*missingCount*: #P-completeness
`Monotone.lean`	238	18	10	one-call reductions; monotone shape, conditional case
`Binary.lean`	206	22	3	`encInstBin`; every result above for it
`Tractable.lean`	1 099	81	26	no constraints; at most k attributes
`ExplicitDegree.lean`	692	46	7	explicit degree, uniform in k
`Restricted.lean`	102	10	6	hardness on Boolean domains, $T_0 = \emptyset$
`Valiant/Gadget.lean`	227	19	14	the graph, ψ, *post*; polynomial time
`Valiant/Counting.lean`	518	38	7	models and independent sets of size K
`Valiant/Pendant.lean`	195	9	0	the model count of ψ
`Valiant/OneCall.lean`	200	14	0	the digit; one-call Valiant
`Valiant/AnyTable.lean`	124	14	2	*countMissingMonotone2*
total	4 520	345	114	

adequacy, the two counting files and the conditional monotone case followed the same day, the binary index and the tractable side on 29 October, and the three remaining results on 31 October, after a written plan (`docs/gaps-plan.md`): counting with at most k attributes, Valiant's theorem and the explicit degree, committed 14, 31 and 35 minutes after the plan. Commit times bound neither when the work on a part began nor the author's time reviewing it. The three parts were developed in parallel sessions.

10.2 What Complexitylib provided and what it lacked

Complexitylib provided the machine model, the encodings, the classes, the Cook–Levin theorem with its parsimonious form, and a large stock of polynomial-time lemmas, among them binary arithmetic (multiplication, division, remainder, powers of two), list folds, the binary list lookup `listGetD_binary`, a fold whose intermediate results grow boundedly (`foldlWithInput_of_bounded_growth`, used for the binary product of the domain sizes), the padding lemma `mem_SharpP_of_polyTime_length`, the bound

Listing 1 The polynomial time of the SAT reduction (`General.lean`). The two `have` lines bring library lemmas for the dense relabelling and the number of variables into the context; `fun_prop` composes them with the lemmas for pairs, lists, `replicate`, `map` and `if`.

```
theorem fromSAT_polyTime : ComputableInPolyTime fromSAT encCNF encInst := by
  have h1 := (CNF.computableInPolyTime_denseRelabel_unary encNat).tidy
  have h2 := SAT.computableInPolyTime_numVars.tidy
  refine .of_tidy ?_
  unfold fromSAT toCond
  fun_prop
```

framework `ComputableTidilyIn0` and promise problems. The development needed five things the library did not have, and worked around each:

1. *Reductions with one oracle call.* The library has many-one, parsimonious and Turing reductions. Definition 1 is defined in `Monotone.lean`, with its composition with parsimonious reductions on either side (`ParsimoniousReducible.trans_oneCall`, `OneCallSharpPHard.trans_parsimonious`).
2. *#P-hardness beyond the SAT family.* The library has #SAT and #3SAT, a decision problem for independent sets, and no counting problem on graphs. The plan was to port Senellart and Gnatenko's chain of four reductions, three of them through counting problems on graphs; the proof instead builds the graph of Section 6 directly as a CNF over numbered vertices and needs no graph library.
3. *Explicit bounds.* The composition lemmas for polynomial time are existential: a composed bound exists, but its degree is not visible. The explicit degree goes through `ComputableTidilyIn0` instead (Section 10.4).
4. *Uniform constants.* The constants of the case-split and constant-output lemmas depend on the machines involved, which is why the per-k machine with constants independent of k is not proved (Section 7).
5. *Reading a list at a binary index.* The first encoding wrote the attribute index in unary, because the list lookup used for the certificate check (`ComputableTidilyInPolyTime.getElem?`) takes a unary position. The binary encoding then used the library's `listGetD_binary`, with the values shifted by one so that the default of an out-of-range lookup matches no value.

The library changes daily and sits on a release-candidate toolchain, so the development pins one commit and one toolchain (Appendix C).

10.3 Polynomial time by automation

Complexitylib registers its polynomial-time lemmas with Mathlib's `fun_prop` tactic, which proves a polynomial-time claim for a term built from registered functions by composing their lemmas. The library's documentation asks that local definitions be unfolded so that `fun_prop` can see their bodies. Most polynomial-time proofs of the development are then three lines, as for the reduction from SAT (Listing 1). The `tidy` forms are machines that restore their tapes, which composition needs; `.of_tidy` returns to the plain notion.

Listing 2 A helper with its composable lemma (`Valiant/Gadget.lean`). The attribute `@[fun_prop]` registers the second theorem, so that `fun_prop` treats `vv` as a primitive.

```
theorem vv_polyTime : ComputableTidilyInPolyTime (fun p : ℕ × Bool => vv p
    .1 p.2)
    (encPair encUnary encBool) encUnary := by
  unfold vv
  fun_prop

@[fun_prop]
theorem vv_polyTime' {α : Type} {input : α ↪ List Bool} {f : α → ℕ} {g :
    α → Bool}
    (hf : ComputableTidilyInPolyTime f input encUnary)
    (hg : ComputableTidilyInPolyTime g input encBool) :
    ComputableTidilyInPolyTime (fun a => vv (f a) (g a)) input encUnary :=
  vv_polyTime.comp (hf.prod hg)
```

For larger constructions, the development does not unfold everything. For the graph of Section 6, whose vertices are numbered by Cantor pairing ($3 \cdot \mathrm{pair}(i, b)$ for the value vertices, and similar codes for occurrences and leaves), the remedy was to give each helper, the three vertex codes and the edge, a lemma in two forms: one for the helper itself, proved by unfolding, and a composable one, registered with `fun_prop`, for the helper applied to any polynomial-time arguments (Listing 2). `fun_prop` then stops at the helper and never unfolds it again.

Automation stops where the size of intermediate values matters. Proofs by hand were needed for the binary product of the domain sizes (a fold whose intermediate products are bounded by the input, through `foldlWithInput_of_bounded_growth`), the enumeration of candidate states (by induction on k), the explicit bounds, and the arithmetic of Valiant's post-processing.

10.4 An instance-dependent time bound

The bound $c\,(|x|+1)^{en+e}$ of Section 7 depends on the instance through its number of attributes n, which is not a function of $|x|$ alone. It is built in three steps.

1. *A class of bounds.* `XPBounded size n T` says that $T(a) \le c\,(\mathit{size}(a)+1)^{e\,(n(a)+1)}$ for some constants (Listing 3). It is closed under sums, constant factors and fixed powers, and constant factors are absorbed into the exponent, since $|x| \ge 1$ (`mul_pow_le_pow_add`).
2. *An explicitly bounded enumeration.* All candidate states of n lists are built by a left fold, `allSections`, whose intermediate states have encodings of length at most $4\,(|L|+1)^{n+2}$ (`length_state_le`); Complexitylib's fold lemma for `ComputableTidilyIn0` then gives `allSections_in0`. Each polynomial-time piece enters with a bound $(|x|+1)^{e}$ (`in0_pow_of_polyTime`), and `pipeline_in0` composes a polynomial-time list builder, the enumeration and a polynomial-time test into one `ComputableTidilyIn0` statement with `XPBounded` time and space.

Listing 3 The class of bounds, and the step to explicit constants (`ExplicitDegree.lean`; statements only). The hypothesis `h` has a hidden constant factor; the conclusion has none.

```
def XPBounded {α : Type} (size n T : α → ℕ) : Prop :=
  ∃ c e, ∀ a, T a ≤ c * (size a + 1) ^ (e * (n a + 1))

theorem timeAndSpace_of_inO_xp {α β : Type} {encIn : α ↪ List Bool} {
    encOut : β ↪ List Bool}
    {f : α → β} {T S n : α → ℕ} (h : ComputableTidilyInO f encIn encOut T
     S)
    (hT : XPBounded (fun a => (encIn a).length) n T)
    (hS : XPBounded (fun a => (encIn a).length) n S) :
    ∃ c d, ComputableTidilyInTimeAndSpace f encIn encOut
      (fun a => c * ((encIn a).length + 1) ^ (d * n a + d))
      (fun a => c * ((encIn a).length + 1) ^ (d * n a + d))
```

3. *Explicit constants.* `timeAndSpace_of_inO_xp` turns such a statement into `ComputableTidilyInTimeAndSpace` with the bound $c\,(|x|+1)^{en+e}$, the form of the theorems.

10.5 Reusable lemmas

Some results are independent of decision tables and could serve other developments on Complexitylib, or be offered upstream; apart from Definition 1 they use only Complexitylib's notions.

- One-call reductions and their compositions with parsimonious ones (`OneCallReducible`, `ParsimoniousReducible.trans_oneCall`, `OneCallSharpPHard.trans_parsimonious`, `SharpPHard.oneCallSharpPHard`).
- One-call #P-hardness of #Monotone-2SAT (`sharpMonotone2SAT_oneCallSharpPHard`), with the parsimonious encoding of the models of any CNF as the independent sets of size K of a graph (`modelCount_eq_card`) and the digit lemma for sums of powers (`digit`).
- The explicit-bound toolkit: `XPBounded` with its closure lemmas, `inO_pow_of_polyTime`, `timeAndSpace_of_inO_xp`, and the bounded enumeration of all sections of a list of lists (`allSections_inO`).
- Size facts for encodings, such as `length_encList_le_of_inDomains`: a list bounded entrywise by another is encoded in no more bits.

10.6 How an AI system was used

The development and this paper were written with an AI system, Claude (Anthropic), used through Claude Code with the model Claude Opus 5.5, between 28 October and 1 November 2026. This subsection records how, from the repository's history; JAR asks that such use be documented in the methods of the paper.

- *Who wrote what.* All 80 commits of the repository up to the revision of this paper were made by the AI system, in ten sessions, each commit message naming its

session. Every Lean file of `LatcComplexity/`, every document of the repository and the text of this paper were drafted by it; `LatcFormal/` is copied from latc-check. The author set the questions, scope and venue, directed every session and approved every sentence of the final text; the Lean code was written by the AI system.

- *Briefs.* Each part of the development was done in its own session from a written task description: the initial development (one session, from the Boolean model to the conditional monotone case), the binary index with the tractable side, each of the three results of the plan `docs/gaps-plan.md`, the monotone count for arbitrary tables, the tightened shape tests, and the shape lemma of Section 4. Each task description was drafted by an AI assistant at the author's direction and approved by the author before its session; the plan and the task descriptions of the sessions started that way are in the repository (`docs/briefs/`), and sessions the author steered directly are recorded in their commit messages.
- *Review of definitions.* Lean checks every proof, so the human review that matters is the review of the trusted definitions, which Appendix B collects. The definitions were checked against latc-check's model and the README by AI referee reads, and the author reviewed their statements. One defect of a definition was found and fixed: the first monotone shape tests accepted conditions on attributes that do not exist, so that a clause $[(0,1),(0,1)]$ over zero attributes passed the test with count 0 and the statement "the all-ones state is valid" failed; the tests now require every attribute index to be declared, and no theorem statement changed. It was found during the development.
- *Iterations on the paper.* The commit messages record four referee reports on an early draft, an independent audit and two further referee reads, each followed by a revision, and a referee read of the JAR version that led to the present revision. All of these reports were produced by AI agents; none was written by a person. The AI system also searched for related work and checked bibliographic records against the publishers' records.
- *Accountability.* The author takes responsibility for the content of the paper and of the development. An AI system does not meet the criteria for authorship and is not an author.

11 Conclusion

For decision tables whose rows are complete tuples, completeness under clause constraints is coNP-complete, finding a missing combination NP-complete, and counting them #P-complete, with machine-checked proofs on Turing machines that apply to latc-check's set-based definitions and hold already for Boolean domains and empty tables. All three are polynomial without constraints; they are also polynomial with a fixed number of attributes, with a degree linear in that number. For monotone two-literal constraints, counting the missing combinations stays #P-complete under one-call reductions, on tables with arbitrary rows and already with none, by a new formal proof of a one-call version of Valiant's theorem in the same model. What remains open in the formalisation is the per-k machine with constants independent of k. The

one-call reductions and the Valiant reduction use only Complexitylib's notions and could be offered upstream.

Supplementary information. The Lean development, latc-complexity, described in Appendix C.

Statements and Declarations

Funding. No funding was received for this work.

Competing interests. The author is a co-author of the 2021 paper that introduced the LATC decision-table completion method, which the author's earlier work [4] re-examines. The author has no other competing interests to declare. (The 2021 paper is [45].)

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. No data were generated or analysed beyond the Lean source code described under Code availability. The decision-table model it builds on is archived as latc-check 0.1.0 (https://doi.org/10.5281/zenodo.23646699).

Materials availability. Not applicable.

Code availability. The Lean development latc-complexity, described in Appendix C, is released as version 1.0 under the MIT licence at https://github.com/carlosalbertorussell/latc-complexity and archived on Zenodo with the DOI https://doi.org/10.5281/zenodo.23860668. The release followed the steps of `RELEASE.md` in the repository.

Author contribution. C.A.R. is the sole author, set the research questions and the task descriptions, reviewed the definitions and the text, and is responsible for the whole work; the Lean proofs and the text were drafted with an AI system, as described in Section 10.6.

Use of generative AI. The use of an AI system is documented in Section 10.6.

Appendix A Main Lean statements

The statements below are copied from the source up to `:=`. Unless noted, they are in the namespace `Latc.PaperD.General`; `Inst` is the type of instances of Section 3.

Bridge (namespace `Latc.PaperD`).

```
theorem holdsAll_eq_eval (S : Prop5.ConSet ℕ) (x : ℕ → Bool) :
    Prop5.holdsAll S x = (toCNF S).eval x
theorem satisfiable_decode_iff (S : Prop5.ConSet ℕ) :
    Prop5.Satisfiable (Prop5.decode S) ↔ (toCNF S).Satisfiable
```

Decision problems and counting.

```
theorem MissingNonempty_npComplete_bin : NPComplete encInstBin
    MissingNonempty
theorem Complete_coNPComplete_bin : coNPComplete encInstBin Complete
theorem missingNonempty_finset_npComplete_bin :
    NPComplete encInstBin {I : Inst | (missing I.con I.table).Nonempty}
theorem complete_finset_coNPComplete_bin :
    coNPComplete encInstBin {I : Inst | missing I.con I.table = ∅}
theorem missingCount_eq_acceptCount (I : Inst) :
    missingCount I = acceptCount cverifier I (certLen I)
theorem sharpSAT_eq_missingCount (φ : CNF ℕ) : SharpSAT φ = missingCount (
    fromSAT φ)
theorem missingCount_sharpPComplete_bin : SharpPComplete encInstBin
    missingCount
```

Boolean domains with empty tables.

```
theorem fromSAT_mem_boolEmpty (φ : CNF ℕ) : fromSAT φ ∈ BoolEmpty
theorem MissingNonemptyBoolEmpty_npHard_bin : NPHard encInstBin
    MissingNonemptyBoolEmpty
theorem CompleteBoolEmpty_coNPHard_bin : coNPHard encInstBin
    CompleteBoolEmpty
theorem missingNonemptyPromise_npHard_bin :
    PromiseProblem.NPHard encInstBin missingNonemptyPromise
theorem missingCountBoolEmpty_sharpPHard_bin : SharpPHard encInstBin
    missingCountBoolEmpty
```

One-call reductions and Valiant. `OneCallReducible` is in `Latc.PaperD.General`; the four theorems after it are in `Latc.PaperD.Valiant`.

```
def OneCallReducible (encA : α ↪ List Bool) (encB : β ↪ List Bool) (f : α
     → ℕ) (g : β → ℕ) :
    Prop :=
  ∃ (r : α → β) (post : α → ℕ → ℕ), ComputableInPolyTime r encA encB ∧
    ComputableInPolyTime (fun p : α × ℕ => post p.1 p.2) (encPair encA
    encNat) encNat ∧
      ∀ a, f a = post a (g (r a))
theorem sharpMonotone2SAT_oneCallSharpPHard :
    OneCallSharpPHard encCNF SharpMonotone2SAT
theorem countValidMonotone2_sharpPComplete_oneCall :
    OneCallSharpPComplete encInstBin countValidMonotone2
theorem countMissingMonotone2_eq_card_missing (I : General.Inst) (h :
    pos2AnyB I = true) :
    countMissingMonotone2 I = (Latc.missing I.con I.table).card
theorem countMissingMonotone2_sharpPComplete_oneCall :
    OneCallSharpPComplete encInstBin countMissingMonotone2
```

Tractability.

```
theorem missingCount_eq_of_noConstraints (I : Inst) (h : I.2.1 = []) :
```

```
    missingCount I = domProd I - distinctRows I
theorem missingCountFixedAttrs_polyTime (k : ℕ) :
    ComputableInPolyTime (fun I : Inst => if I.n ≤ k then missingCount I
    else 0)
      encInstBin encNat
theorem missingNonempty_explicitDegree :
    ∃ c d, ComputableTidilyInTimeAndSpace
      (fun I : Inst => decide (missing I.con I.table).Nonempty) encInstBin
    encBool
      (fun I => c * ((encInstBin I).length + 1) ^ (d * I.n + d))
      (fun I => c * ((encInstBin I).length + 1) ^ (d * I.n + d))
theorem missingNonemptyFixedAttrs_uniform :
    ∃ c d₀ d₁, ∀ k, ComputableTidilyInTimeAndSpace
      (fun I : {I : Inst // I.n ≤ k} => decide (missing I.1.con I.1.table).
    Nonempty)
      ((Function.Embedding.subtype _).trans encInstBin) encBool
      (fun I => c * ((encInstBin I.1).length + 1) ^ (d₀ * k + d₁))
      (fun I => c * ((encInstBin I.1).length + 1) ^ (d₀ * k + d₁))
```

The unary forms. Each polynomial-time result of Section 7 also holds for `encInst`: `completeNoConstraints_mem_P_unary`, `missingNonemptyNoConstraints_mem_P_unary`, `missingCountNoConstraints_polyTime_unary`, `completeFixedAttrs_mem_P_unary`, `missingNonemptyFixedAttrs_mem_P_unary` and `missingCountFixedAttrs_polyTime_unary`, the first five through `mem_P_of_mem_P_bin` or the re-encoding of Section 3. The hardness results have their unary forms without the suffix `_bin` (`MissingNonempty_npComplete`, `Complete_coNPComplete`, `missingNonempty_finset_npComplete`, `complete_finset_coNPComplete`, `missingCount_sharpPComplete`), and the monotone ones with the suffix `_unary` (`countValidMonotone2_sharpPComplete_oneCall_unary`, `countMissingMonotone2_sharpPComplete_oneCall_unary`).

Appendix B Trusted definitions

The definitions below are the ones a reader must trust beyond Complexitylib, CSLib and Mathlib (Section 9). They are copied verbatim from the source, with their documentation comments, file by file. Namespaces: `Latc` for latc-check's model, `Latc.PaperD.General` for the rest except `pos2AnyB` and `countMissingMonotone2`, which are in `Latc.PaperD.Valiant`.

latc-check's model (`LatcFormal/Basic.lean`). Here ι is a finite type of attributes, `D i` a finite domain for each, and `C` a decidable predicate on tuples.

```
/-- The valid space 𝒱 = {x ∈ 𝒟 : c(x) for all c ∈ C} (Section 3.1). -/
def validSpace : Finset (∀ i, D i) := univ.filter C

/-- The missing set M(T₀) = 𝒱 \ cond(T₀) (Section 3.2). -/
def missing (T : Finset (∀ i, D i)) : Finset (∀ i, D i) := validSpace C \ T
```

*Instances and the decision problems (**General.lean**).*

```
/-- A condition A_i = v, as the pair `(i, v)`. -/
abbrev Cond := ℕ × ℕ

/-- An instance: domain sizes, clause constraints, and the condition tuples
    of T_0. -/
abbrev Inst := List ℕ × (List (List Cond) × List (List ℕ))

/-- Instances: binary domain sizes, values and rows; unary attribute indices
    . -/
abbrev encInst : Inst ↪ List Bool :=
  encPair (encList encNat) (encPair (encList (encList (encPair encUnary
    encNat)))
    (encList (encList encNat)))

/-- `s` is a state of the declared domains `d`: one value per attribute,
    inside its domain. -/
def InDomains (d s : List ℕ) : Prop := List.Forall₂ (· < ·) s d

/-- The state `s` satisfies the condition A_i = v. -/
def CondHolds (s : List ℕ) (a : Cond) : Prop := s[a.1]? = some a.2

/-- The state `s` satisfies every clause constraint of `S`. -/
def Valid (S : List (List Cond)) (s : List ℕ) : Prop := ∀ c ∈ S, ∃ a ∈ c,
    CondHolds s a

/-- M(T_0) is non-empty: some valid state of the declared domains is not a
    condition tuple
of T_0. -/
def MissingNonempty : Set Inst :=
  {I | ∃ s, InDomains I.1 s ∧ Valid I.2.1 s ∧ s ∉ I.2.2}

/-- T_0 is complete: M(T_0) = ∅. -/
def Complete : Set Inst := MissingNonempty^c
```

*The binary encoding (**Binary.lean**).*

```
/-- Instances with every number in binary, the attribute index of a
    condition included. -/
abbrev encInstBin : Inst ↪ List Bool :=
  encPair (encList encNat) (encPair (encList (encList (encPair encNat encNat
    )))
    (encList (encList encNat)))
```

*The table an instance denotes (**Adequacy.lean**).*

```
/-- The number of attributes. -/
abbrev Inst.n (I : Inst) : ℕ := I.1.length

/-- The declared domain of attribute `i`: the values `0, ..., d_i - 1`. -/
```

```
abbrev Inst.Dom (I : Inst) (i : Fin I.n) : Type := Fin (I.1.get i)

/-- The list of values of a condition tuple. -/
def Inst.toList (I : Inst) (x : ∀ i, I.Dom i) : List ℕ := List.ofFn fun i =>
     (x i).val

/-- The constraint predicate of the instance, on condition tuples. -/
def Inst.con (I : Inst) (x : ∀ i, I.Dom i) : Prop := Valid I.2.1 (I.toList x
    )

/-- The condition set of T₀: the tuples whose list of values is a row of `T
    `. -/
def Inst.table (I : Inst) : Finset (∀ i, I.Dom i) :=
  Finset.univ.filter fun x => I.toList x ∈ I.2.2
```

Counting (`CountingGeneral.lean`).

```
def missingCount (I : Inst) : ℕ := (missing I.con I.table).card
```

One-call reductions and the monotone problems (`Monotone.lean`).

```
/-- `f` reduces to `g` with one oracle call: `f a = post a (g (r a))` for
    polynomial-time `r` and
`post`, the oracle answer read in binary. -/
def OneCallReducible (encA : α ↪ List Bool) (encB : β ↪ List Bool) (f : α
     → ℕ) (g : β → ℕ) :
    Prop :=
  ∃ (r : α → β) (post : α → ℕ → ℕ), ComputableInPolyTime r encA encB ∧
    ComputableInPolyTime (fun p : α × ℕ => post p.1 p.2) (encPair encA
    encNat) encNat ∧
      ∀ a, f a = post a (g (r a))

/-- Every function in `#P`, with any encoding of its instances, reduces to `
    g` with one call. -/
def OneCallSharpPHard (encB : β ↪ List Bool) (g : β → ℕ) : Prop :=
  ∀ {α : Type} (enc : α ↪ List Bool) (f : α → ℕ), f ∈ SharpP enc →
    OneCallReducible enc encB f g

/-- `g` is in `#P` and one-call `#P`-hard. -/
def OneCallSharpPComplete (encB : β ↪ List Bool) (g : β → ℕ) : Prop :=
  g ∈ SharpP encB ∧ OneCallSharpPHard encB g

/-- Every clause has two literals, both positive: `xᵢ ∨ xⱼ`. -/
def monotone2B (φ : CNF ℕ) : Bool := φ.all fun c => c.length == 2 && c.all
    (·.2)

/-- `#Monotone-2SAT`: the model count of a monotone two-literal formula;
    zero on other syntax. -/
def SharpMonotone2SAT (φ : CNF ℕ) : ℕ := if monotone2B φ then SharpSAT φ
    else 0
```

```
/-- Domains {0, 1}, clauses of two conditions with value 1 on declared
    attributes (index below
the number of attributes), and T₀ = ∅. -/
def pos2B (I : Inst) : Bool :=
  I.1.all (· == 2) &&
      I.2.1.all (fun c =>
        c.length == 2 && c.all (fun a => decide (a.1 < I.1.length) && a.2 ==
     1)) &&
    I.2.2.isEmpty

/-- |𝒱| for clause constraints of two conditions A_i = 1, A_j = 1 over domains
     {0, 1}, in
latc-check's model (‘missingCount‘ with T₀ = ∅); zero on other syntax. -/
noncomputable def countValidMonotone2 (I : Inst) : ℕ := if pos2B I then
    missingCount I else 0
```

The monotone count for any table (`Valiant/AnyTable.lean`).

```
/-- Domains {0, 1} and clauses of two conditions with value 1 on declared
    attributes (index below
the number of attributes); any table T₀. -/
def pos2AnyB (I : General.Inst) : Bool :=
  I.1.all (· == 2) &&
    I.2.1.all (fun c =>
      c.length == 2 && c.all (fun a => decide (a.1 < I.1.length) && a.2 ==
    1))

/-- |M(T₀)| for clause constraints of two conditions A_i = 1, A_j = 1 over
    domains {0, 1}, any
table T₀, in latc-check's model; zero on other syntax. -/
noncomputable def countMissingMonotone2 (I : General.Inst) : ℕ :=
  if pos2AnyB I then missingCount I else 0
```

Boolean domains with empty tables (`Restricted.lean`).

```
/-- Boolean domains and an empty table: every declared domain size is 2 and
    T₀ has no rows. -/
def BoolEmpty : Set Inst := {I | (∀ d ∈ I.1, d = 2) ∧ I.2.2 = []}

/-- M(T₀) ≠ ∅, on Boolean domains with T₀ = ∅. -/
def MissingNonemptyBoolEmpty : Set Inst := MissingNonempty ∩ BoolEmpty

/-- M(T₀) = ∅, on Boolean domains with T₀ = ∅. -/
def CompleteBoolEmpty : Set Inst := Complete ∩ BoolEmpty

/-- The promise problem: on Boolean domains with T₀ = ∅, tell M(T₀) ≠ ∅ from
     M(T₀) = ∅. -/
def missingNonemptyPromise : PromiseProblem Inst :=
  (PromiseProblem.ofSet MissingNonempty).restrict BoolEmpty
```

```
/-- |M(T₀)| on Boolean domains with T₀ = ∅, and `0` on other instances. -/
def missingCountBoolEmpty (I : Inst) : ℕ := if I ∈ BoolEmpty then
    missingCount I else 0
```

Appendix C Artifact and reproducibility

Contents. The development is the directory `LatcComplexity/` of the repository latc-complexity: 4 903 lines of Lean in 17 files, 4 520 without the two axiom-audit files (Table 3), on top of latc-check's model in `LatcFormal/`. The eleven files of `LatcFormal/` and its root file are byte-identical to the folder `formal/` of the archived release latc-check 0.1.0 [5]. The README lists every main theorem with its statement, and `docs/paper-facts.md` records the fully qualified name, file and line of every name this paper cites.

The Boolean model. The repository also contains the first form of the development, a Boolean model (`Bridge.lean`, `Counting.lean`): its attributes are the variables of a Complexitylib CNF formula used as the constraints, and its rows are bit vectors. NP-, coNP- and #P-completeness are proved for it too (`Latc.PaperD.MissingNonempty_npComplete`, `Latc.PaperD.Complete_coNPComplete`, `Latc.PaperD.countMissing_sharpPComplete`), the last with a parsimonious reduction *to* #SAT (`countMissing_eq_sharpSAT`). It has no adequacy theorem, and the results of this paper do not use it.

Versions. Lean `v4.36.0-rc1`; Complexitylib at commit `b7acfc2` (28 October 2026), with the CSLib and Mathlib it pins. The earlier, pre-CSLib version of Complexitylib, which Russell [4] cites, is a separate repository with a different API.

Build and audit. The build reported here is a cold build of the released version (the tag `v1.0`, whose Lean files are those of this paper), made on 1 November 2026 on a Linux x86-64 machine with four cores, in a fresh clone with no `~/.elan`, `.lake` or Mathlib cache, with Lean `4.36.0-rc1`, Lake 5.0.0 and elan 4.2.4. `lake exe cache get` downloaded the toolchain and all 9 047 precompiled Mathlib files in 97 s. `lake build` compiled 239 modules of CSLib and Complexitylib and the 30 modules of the repository in 213 s and ended with `Build completed successfully (18651 jobs)`, exit code 0, no warnings and no errors; the checkout, toolchain and caches took 11.4 GB of disk. The axiom audit, `lake env lean` on the three audit files (`LatcComplexity/Axioms.lean`, `LatcComplexity/Valiant/Axioms.lean` and `LatcFormal/Axioms.lean`), took 14 s and printed 397 answers: 344 use all three standard axioms, 51 use one or two of them, 2 use none, and none uses `sorryAx`. A whole-word search of the `.lean` files finds `sorry`, `admit`, `native_decide` and `axiom` only in the `#print axioms` commands.

Release. The file `RELEASE.md` lists the steps of the release, carried out on 1 November 2026: a cold build and the full audit, tagging version 1.0, making the repository public, and archiving the tag on Zenodo (https://doi.org/10.5281/zenodo.23860668). The commit-by-commit history of earlier builds is in the README.

References

[1] Pollack, S.L., Hicks, H.T. Jr., Harrison, W.J.: Decision Tables: Theory and Practice. Wiley-Interscience, New York (1971)

[2] Suwa, M., Scott, A.C., Shortliffe, E.H.: An approach to verifying completeness and consistency in a rule-based expert system. AI Magazine **3**(4), 16–21 (1982) https://doi.org/10.1609/aimag.v3i4.377

[3] Calvanese, D., Dumas, M., Laurson, Ü., Maggi, F.M., Montali, M., Teinemaa, I.: Semantics and analysis of DMN decision tables. In: Rosa, M.L., Loos, P., Pastor, O. (eds.) Business Process Management – 14th International Conference, BPM 2016, Rio de Janeiro, Brazil, September 18–22, 2016, Proceedings. Lecture Notes in Computer Science, vol. 9850, pp. 217–233. Springer, ??? (2016). https://doi.org/10.1007/978-3-319-45348-4_13

[4] Russell, C.A.: Where Declared Domains Are Lost: LATC, DMN Analysers and Public DMN Models. Under review at Data & Knowledge Engineering (2026)

[5] Russell, C.A.: Latc_check: Independent Checks for Finite-domain Decision-table Completion. https://doi.org/10.5281/zenodo.23646699 . https://doi.org/10.5281/zenodo.23646699

[6] Schlesinger, S., The Complexitylib contributors: Complexitylib. Lean 4 library built on CSLib; commit b7acfc2 of 28 October 2026. https://github.com/cslib-community/complexitylib

[7] Hamilton, A., Roughan, M., Nguyen, G.T.: Boolean Expressions in Firewall Analysis (2022). https://doi.org/10.48550/arXiv.2205.04210 . https://arxiv.org/abs/2205.04210

[8] Vu, D.N., Demetrovics, J.: Labeled null semantics for incomplete decision tables: Certain, possible, and conditional decision reasoning. Algorithms **19**(9), 725 (2026) https://doi.org/10.3390/a19090725

[9] Barrett, C., Chaudhuri, S., Montesi, F., Grundy, J., Kohli, P., Moura, L., Rademaker, A., Yingchareonthawornchai, S.: CSLib: The Lean Computer Science Library (2026). https://arxiv.org/abs/2602.04846

[10] Valiant, L.G.: The complexity of computing the permanent. Theoretical Computer Science **8**(2), 189–201 (1979) https://doi.org/10.1016/0304-3975(79)90044-6

[11] Krentel, M.W.: The complexity of optimization problems. Journal of Computer and System Sciences **36**(3), 490–509 (1988) https://doi.org/10.1016/0022-0000(88)90039-6

[12] Valiant, L.G.: The complexity of enumeration and reliability problems. SIAM Journal on Computing **8**(3), 410–421 (1979) https://doi.org/10.1137/0208032

[13] Zankó, V.: #P-completeness via many-one reductions. International Journal of Foundations of Computer Science **2**(1), 77–82 (1991) https://doi.org/10.1142/S0129054191000066

[14] Creignou, N., Hermann, M.: Complexity of generalized satisfiability counting problems. Information and Computation **125**(1), 1–12 (1996) https://doi.org/10.1006/inco.1996.0016

[15] Senellart, P.: descriptive-complexity: a Lean library of descriptive complexity. Software, commit fc068f6 (21 October 2026); concept DOI 10.5281/zenodo.21678423 (2026). https://doi.org/10.5281/zenodo.21678423 . https://github.com/PierreSenellart/descriptive-complexity

[16] Papadimitriou, C.H., Yannakakis, M.: On the complexity of database queries. Journal of Computer and System Sciences **58**(3), 407–427 (1999) https://doi.org/10.1006/jcss.1999.1626

[17] Cygan, M., Fomin, F.V., Kowalik, Ł., Lokshtanov, D., Marx, D., Pilipczuk, M., Pilipczuk, M., Saurabh, S.: Parameterized Algorithms. Springer, ??? (2015). https://doi.org/10.1007/978-3-319-21275-3

[18] Cook, S.A.: The complexity of theorem-proving procedures. In: Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC '71), pp. 151–158. ACM, ??? (1971). https://doi.org/10.1145/800157.805047

[19] Elmallah, E.S., Gouda, M.G.: Hardness of firewall analysis. IEEE Transactions on Dependable and Secure Computing **14**(3), 339–349 (2017) https://doi.org/10.1109/TDSC.2015.2455532

[20] Thiemann, R., Yamada, A.: A verified algorithm for deciding pattern completeness with optimal asymptotic complexity. Journal of Logical and Algebraic Methods in Programming **151**, 101129 (2026) https://doi.org/10.1016/j.jlamp.2026.101129

[21] Kapur, D., Narendran, P., Rosenkrantz, D.J., Zhang, H.: Sufficient-completeness, ground-reducibility and their complexity. Acta Informatica **28**(4), 311–350 (1991) https://doi.org/10.1007/BF01893885

[22] Chan, T.M.: A (slightly) faster algorithm for Klee's measure problem. Computational Geometry **43**(3), 243–250 (2010) https://doi.org/10.1016/j.comgeo.2009.01.007

[23] Bringmann, K., Friedrich, T.: Approximating the volume of unions and intersections of high-dimensional geometric objects. Computational Geometry **43**(6–7),

601–610 (2010) https://doi.org/10.1016/j.comgeo.2010.03.004

[24] Vanthienen, J., Mues, C., Aerts, A.: An illustration of verification and validation in the modelling phase of KBS development. Data & Knowledge Engineering **27**(3), 337–352 (1998) https://doi.org/10.1016/S0169-023X(98)80003-7

[25] Heitmeyer, C.L., Jeffords, R.D., Labaw, B.G.: Automated consistency checking of requirements specifications. ACM Transactions on Software Engineering and Methodology **5**(3), 231–261 (1996) https://doi.org/10.1145/234426.234431

[26] Heimdahl, M.P.E., Leveson, N.G.: Completeness and consistency in hierarchical state-based requirements. IEEE Transactions on Software Engineering **22**(6), 363–377 (1996) https://doi.org/10.1109/32.508311

[27] Calvanese, D., Dumas, M., Laurson, Ü., Maggi, F.M., Montali, M., Teinemaa, I.: Semantics, analysis and simplification of DMN decision tables. Information Systems **78**, 112–125 (2018) https://doi.org/10.1016/j.is.2018.01.010

[28] Calvanese, D., Montali, M., Dumas, M., Maggi, F.M.: Semantic DMN: Formalizing and reasoning about decisions in the presence of background knowledge. Theory and Practice of Logic Programming **19**(4), 536–573 (2019) https://doi.org/10.1017/S1471068418000479

[29] Owre, S., Rushby, J., Shankar, N.: Integration in PVS: Tables, types, and model checking. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS '97). Lecture Notes in Computer Science, vol. 1217, pp. 366–383. Springer, ??? (1997). https://doi.org/10.1007/BFb0035400

[30] Menghi, C., Balai, E., Valovcin, D., Sticksel, C., Rajhans, A.: Completeness and consistency of tabular requirements: An SMT-based verification approach. IEEE Transactions on Software Engineering **51**(2), 595–620 (2025) https://doi.org/10.1109/TSE.2025.3530820

[31] Zanenga, A.F., Bisceglia, N.M., Ippoliti, B., Bombarda, A., Gargantini, A., Rajhans, A., Menghi, C.: Theano: A tool for verifying the consistency and completeness in tabular requirements. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), pp. 1148–1152. ACM, ??? (2025). https://doi.org/10.1145/3696630.3728599

[32] Levy, A.Y., Rousset, M.-C.: Verification of knowledge bases based on containment checking. In: Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI-96), pp. 585–590. AAAI Press, ??? (1996). https://aaai.org/Papers/AAAI/1996/AAAI96-087.pdf

[33] Levy, A.Y., Rousset, M.-C.: Verification of knowledge bases based on containment checking. Artificial Intelligence **101**(1–2), 227–250 (1998) https://doi.org/10.1016/S0004-3702(98)00021-6

[34] Gamboa, R., Cowles, J.: A mechanical proof of the Cook-Levin theorem. In: Slind, K., Bunker, A., Gopalakrishnan, G. (eds.) Theorem Proving in Higher Order Logics, 17th International Conference, TPHOLs 2004, Park City, Utah, USA, September 14-17, 2004, Proceedings. Lecture Notes in Computer Science, vol. 3223, pp. 99–116. Springer, ??? (2004). https://doi.org/10.1007/978-3-540-30142-4_8

[35] Gäher, L., Kunze, F.: Mechanising Complexity Theory: The Cook-Levin Theorem in Coq. In: Cohen, L., Kaliszyk, C. (eds.) 12th International Conference on Interactive Theorem Proving (ITP 2021). Leibniz International Proceedings in Informatics (LIPIcs), vol. 193, pp. 20–12018. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021). https://doi.org/10.4230/LIPIcs.ITP.2021.20

[36] Balbach, F.J.: The Cook-Levin theorem. Archive of Formal Proofs (2023). https://isa-afp.org/entries/Cook_Levin.html, Formal proof development

[37] Automatic Refinements and Polynomial-Time Reductions in Isabelle/HOL. https://github.com/rosskopfs/poly-reductions. Formerly Karp21; commit d741d95

[38] Senellart, P., Gnatenko, A.: Descriptive Complexity in Lean: Completeness by First-Order Reductions (2026). https://arxiv.org/abs/2609.18261

[39] The mathlib Community: The Lean mathematical library. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), pp. 367–381. ACM, ??? (2020). https://doi.org/10.1145/3372885.3373824

[40] Spelier, P., Gent, D.: The file `Mathlib/Computability/TuringMachine/Computable.lean`. File of Mathlib, at commit 625d8914199bc567cca9b91f90e8e5afafa5ae51 (tag v4.36.0-rc1) (2020). https://github.com/leanprover-community/mathlib4/blob/625d8914199bc567cca9b91f90e8e5afafa5ae51/Mathlib/Computability/TuringMachine/Computable.lean

[41] Xu, J., Zhang, X., Urban, C.: Mechanising Turing machines and computability theory in Isabelle/HOL. In: Blazy, S., Paulin-Mohring, C., Pichardie, D. (eds.) Interactive Theorem Proving (ITP 2013). Lecture Notes in Computer Science, vol. 7998, pp. 147–162. Springer, ??? (2013). https://doi.org/10.1007/978-3-642-39634-2_13

[42] Forster, Y., Kunze, F., Wuttke, M.: Verified programming of Turing machines in Coq. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), pp. 114–128. ACM, ??? (2020). https://doi.org/10.1145/3372885.3373816

[43] Carneiro, M.: Formalizing Computability Theory via Partial Recursive Functions. In: Harrison, J., O'Leary, J., Tolmach, A. (eds.) 10th International Conference

on Interactive Theorem Proving (ITP 2019). Leibniz International Proceedings in Informatics (LIPIcs), vol. 141, pp. 12–11217. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2019). https://doi.org/10.4230/LIPIcs.ITP.2019.12

[44] Forster, Y., Kunze, F., Smolka, G., Wuttke, M.: A mechanised proof of the time invariance thesis for the weak call-by-value λ-calculus. In: 12th International Conference on Interactive Theorem Proving (ITP 2021). Leibniz International Proceedings in Informatics (LIPIcs), vol. 193, pp. 19–11920. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, ??? (2021). https://doi.org/10.4230/LIPIcs.ITP.2021.19

[45] Raggio, R.A., Russell, C.A.: Business rules (BR); expert systems (KBS); LATC algorithm: A proposal for functional integration. International Journal of Scientific & Engineering Research **12**(1), 576–596 (2021)